# AI Invoice Auditor

## 60 viva questions with model answers

### Four-member preparation guide

Focus: added ADK/A2A, MCP, Responsible AI and Langfuse layers, plus the implemented review extension. Baseline topics are included only where they help explain those additions.

### How to answer

Start with the short answer, point to the code boundary, explain what evidence demonstrates it, and state the important limitation. These are preparation answers, not a claim that every runtime test has passed.

### Speaking allocation

| Member | Primary contribution |
|---|---|
| Anshuman Shukla | Google ADK and A2A delegation |
| Aman Patidar | MCP discovery and tool contracts |
| Ayush Gupta | RAI controls and risk-gated indexing |
| Param Srivastava | Langfuse and review integration |

### Important distinctions

Six containers are not seven agent cards or thirteen tools. Discovery is not execution. Fallback success is not live Bedrock proof. Feedback history is not complete immutable report versioning. A trace is not a privacy guarantee.

Use PRESENTATION_SCRIPT.txt for the actual five-slide speaking order. Section answers below are intentionally more detailed than slide text.

# Shared foundations

## All four members

Pehle baseline aur addition ka difference clear rakho. Six services, seven agent cards and thirteen tools are different counts.

### Q01. What is the business problem?

Vendor invoices arrive in different formats and languages. The system extracts evidence, compares billed values with ERP references, produces findings and supports human review and evidence-based questions. It does not execute payments.

### Q02. Which capabilities are genuinely new over the old specification?

The main added layers are Google ADK with A2A delegation, discoverable MCP tools, explicit Responsible AI controls and Langfuse observability. The weighted indexing risk gate is an additional policy; implemented revalidation extends an old optional requirement.

### Q03. What was already required last month?

OCR and multilingual extraction, translation, ERP validation, reports, five-role RAG, triad reflection, Streamlit review, persistent feedback, PostgreSQL, Qdrant, Redis and the six-service Docker deployment. Do not present these alone as newly introduced features.

### Q04. How many services, agent cards and tools exist?

Compose defines six services. The inspected source registers five LangGraph-labeled and two ADK-labeled agent cards, and thirteen tool decorators. Some README comments still say eleven tools; the runtime directories show what actually registered.

### Q05. What is the difference between a container, agent, node and tool?

A container is a deployment runtime. An agent is a logical capability or decision role. A node is a graph step. A tool implements a reusable operation. Several nodes and agent roles can share one container.

### Q06. Explain the principal storage responsibilities.

PostgreSQL holds authoritative current audit/reference data, feedback and checkpoints. Qdrant holds derived searchable vectors. Redis caches recomputable results. Original documents and report files remain on mounted filesystem paths.

### Q07. What is the normal processing order?

Extractor, translator, parser, validator, delegated business validation, reporter, risk gate and conditional indexing. Only the auto-approved branch automatically indexes a fresh audit in the normal graph.

### Q08. What is the difference between recommendation and review status?

Recommendation expresses the reporter outcome, such as approve, manual_review or reject. Review status controls operational routing: auto_approved, pending_review, approved or rejected. They are calculated at different boundaries and should both be inspected.

### Q09. What would prove the project works rather than merely starts?

Show actual extraction and findings, correct reference comparison, a real delegation/tool path, the selected execution backend, reviewer behavior, indexed evidence and exported traces. A healthy API, an installed SDK or a directory entry is not enough.

### Q10. Which implementation should we explain?

Use the current Anshuman project. Do not transfer the earlier separate implementation's 76-test result, Groq UI, ChatBedrockConverse initialization or immutable report-version architecture to this codebase.

# Part 1: Google ADK and A2A

**Anshuman Shukla**

Skill delegate hoti hai, Task result aata hai. Card metadata is not proof of a live ADK turn.

### Q11. Why add Google ADK if LangGraph already orchestrates the workflow?

LangGraph continues to coordinate stages. ADK provides a distinct tool-using agent execution framework for translation and business validation. The addition demonstrates cross-framework cooperation rather than replacing the original workflow.

### Q12. What does A2A add?

A named skill/message/task boundary between agent roles. A caller can request a capability without depending directly on the other framework's implementation details, then consume a structured result or failure.

### Q13. What is an AgentCard?

Metadata describing an agent's name, framework, address, capabilities and skills. It helps a caller inspect what can be requested. A card advertises capability; it does not prove successful model execution or establish authentication.

### Q14. Trace one translation request.

workflow.translation_node calls the A2A translation-agent skill translate_invoice. The registered ADK executor attempts a runner turn and uses lang_bridge through an MCP FunctionTool wrapper. The structured translation result is copied back into workflow state.

### Q15. What does the ADK base class build?

An LlmAgent with a LiteLlm model and FunctionTool wrappers, an InMemorySessionService and a Runner. The runner consumes a user message and reads a final response. ADK conversational sessions differ from PostgreSQL workflow checkpoints.

### Q16. Does a google.genai import mean Gemini is being called?

Not by itself. ADK uses its message types here, while the intended model is selected by LiteLlm with the configured Bedrock model string. Judge actual routing from configuration and execution evidence, not only import names.

### Q17. Does A2A always make an HTTP request?

No. Empty A2A_BASE_URL uses custom in-process dispatch. A configured base URL uses HTTP JSON-RPC. The same logical request shape is intended for both, but a default task does not imply a network hop.

### Q18. What happens when ADK cannot run?

The agent can use run_deterministic and invoke the same MCP capability directly. A higher-level A2A failure can also trigger a workflow fallback. Inspect executor and logs rather than claiming every successful task used ADK reasoning.

### Q19. How are A2A results represented?

The custom models represent Message parts and Task status/artifacts. The client sends message/send, verifies a completed task and extracts returned artifact data. Task IDs, invoice IDs and context/thread IDs serve different purposes.

### Q20. Is this an official A2A SDK conformance demonstration?

No. The source implements its own models, registry, server and client. Describe it as a custom A2A-shaped interoperability layer. Official conformance, transport compatibility and secure authentication require separate evidence.

# Part 2: MCP tool discovery

**Aman Patidar**

Registry se capability discover karo, phir named call ka result/error explain karo. Metadata is not automatic enforcement.

### Q21. How does MCP differ from A2A?

MCP exposes tools that an agent uses to perform operations. A2A delegates a skill or task to another agent role. An A2A-invoked ADK agent may itself call an MCP tool.

### Q22. How does a function become a discoverable tool?

The registry.tool decorator creates a Tool definition containing name, description, input-schema metadata, handler and annotations. Importing the module registers it, and bootstrap ensures the required modules load once.

### Q23. What does tools/list return?

Tool metadata such as names, descriptions, inputSchema and ownership/read-only/guardrail hints. Discovery lets clients inspect available capabilities without invoking the underlying business operation.

### Q24. Walk through tools/call.

The MCP client verifies the named tool is advertised, builds a request with arguments and sends it through in-process or HTTP dispatch. The server calls ToolRegistry, executes the handler and shapes structured content or an error for the client.

### Q25. Name representative invoice and RAG tools.

Invoice examples are data_harvester, lang_bridge, business_validation and insight_reporter. RAG examples are vector_indexer, semantic_retriever, chunk_ranker, response_synthesizer and rag_evaluator. erp_lookup and query_refiner are additional helpers.

### Q26. Why thirteen tools when some documentation says eleven?

The current decorators include eleven primary capabilities plus erp_lookup and query_refiner. Static comments can become stale; compare the source inventory with the actual /mcp/tools response after bootstrap.

### Q27. Does inputSchema guarantee complete argument validation?

No. This registry advertises schemas but directly calls handler(**arguments), relying on signatures and handler-specific checks. It shapes TypeError and other failures; that is not the same as comprehensive JSON Schema enforcement.

### Q28. Are all graph nodes invoked through MCP?

No. Active RAG roles and ADK wrappers use the MCP client, while some ordinary invoice nodes call Python functions directly. Explain specific call chains instead of claiming the whole codebase exclusively uses MCP.

### Q29. How are tool failures handled?

The registry produces a uniform ok/error payload. The server can return isError content, and MCPClient.call raises MCPToolError. try_call logs the failure and returns a configured default for optional work. This does not imply transaction rollback.

### Q30. Do a guardrail annotation and readOnlyHint enforce safety?

No. They communicate metadata. Actual handler scan calls and write behavior determine enforcement. Review the implementation before assuming a named guardrail or read-only hint prevents unsafe effects.

# Part 3: Responsible AI and risk gate

**Ayush Gupta**

Input/output safety, RAG quality aur risk routing alag controls hain. Their thresholds have different meanings.

### Q31. What is prompt injection in an invoice system?

Untrusted document or query text attempts to override the audit task, for example by instructing the system to ignore validation and approve a bill. It must be treated as evidence data, not as a privileged system instruction.

### Q32. Why not discard every document containing hostile text?

That could let a sender evade audit simply by inserting an attack phrase. The document scan records findings and fences the content as untrusted while allowing the invoice to remain subject to audit.

### Q33. What controls apply to user queries?

Configured length limits, instruction-override detection, supported scope checks and secret stripping. An explicit blocked query may terminate before retrieval. These regex controls do not detect every possible irrelevant or malicious request.

### Q34. What controls apply to model answers?

Supported secret/PII stripping, a lexical grounding floor and an advisory human-accountability disclaimer for certain decision language. An insufficiently supported answer can be replaced with a refusal.

### Q35. Are these AWS managed Bedrock Guardrails?

No. The visible controls are application-level regex and lexical logic in guardrails.py and YAML policy. A separate AWS managed guardrail service is not established merely by naming this module guardrails.

### Q36. How does the risk gate decide whether to index?

It sums configured discrepancy severity and additional error/confidence weights, then applies hard blocks. Without a hard block, score at or below 0.30 can auto-approve. Otherwise the normal fresh-processing path holds the record for human review.

### Q37. Is risk_score=0.60 a sixty-percent fraud probability?

No. It is a weighted rule sum. One ERROR commonly contributes 0.60; a score can exceed one. It is not trained or calibrated as a probability of fraud.

### Q38. Explain the three different 0.30/0.70 concepts.

The risk gate default threshold is 0.30 for review routing. The answer lexical groundedness floor is also 0.30 but checks text support. The RAG triad average must be strictly greater than 0.70. Equal numbers do not mean equal semantics.

### Q39. What important edge cases remain in risk routing?

Confidence uses value or 1.0, so exact 0.0 can become 1.0 in that expression. Data errors may also contribute through both discrepancy and flat-error weights. A one-WARNING score at 0.30 can auto-approve if no hard block fires.

### Q40. Does the risk gate guarantee no questionable record can ever be searched?

No. It controls the normal fresh-processing branch. Direct indexing APIs exist, and rejecting a previously indexed record does not explicitly remove all old vectors. Workflow policy is not complete authorization or revocation.

# Part 4: Langfuse and human review

**Param Srivastava**

Trace execution ko explain karta hai; audit DB decision ko. Feedback, revalidation aur approval separate operations hain.

### Q41. How does tracing differ from the audit database?

The audit database stores business evidence and review outcomes. Tracing exposes execution path, timing, errors and fallbacks. A trace helps debug an outcome but does not independently prove its financial correctness.

### Q42. Explain trace, span, generation and score.

A trace represents one logical run. A span is an operation nested inside it. A generation represents a model-call observation. A score attaches a metric, such as RAG context relevance, to execution evidence.

### Q43. What does /observability/test prove?

It exercises the tracing facade using a synthetic trace, nested span, ping/pong generation and test score. Locate the exported observation in the configured project. It does not make a real Bedrock invoice-generation call.

### Q44. Is model token/cost coverage complete?

Do not claim that. record_generation is defined and explicitly used by the synthetic self-test; the wrapper does not universally invoke it for every completion. Trace/span visibility is broader than proven per-model-call cost accounting.

### Q45. What happens if Langfuse is not configured?

The observability facade can degrade to structured logging. The application may remain operational, but cloud trace export has not been demonstrated. SDK installation alone is insufficient.

### Q46. Are trace payloads guaranteed anonymous?

No. The helpers accept input/output payloads and may include invoice or query content. Truncation limits length, not sensitivity. Selected guardrails sanitize some paths; they are not universal anonymization.

### Q47. How do feedback and revalidation differ?

Feedback inserts a correction suggestion/history record. Revalidation reconstructs current state, applies allowed changes, reruns validation/reporting and recalculates risk. Recording feedback alone does not automatically apply it.

### Q48. What is the exact revalidation order?

The endpoint runs validator_node, then reporter_node, then compute_risk, and explicitly keeps review_status pending_review for the reviewer. It does not execute the full original eight-node graph or every extra ADK delegation again.

### Q49. Are all original fields and report versions immutable?

No. Current invoice headers/extracted_raw are upserted, line records can be replaced and named report files can be overwritten. Separate feedback retains changes, but it is not complete append-only extraction/report-version storage.

### Q50. What do approve and reject actually do?

Approve requests indexing, records approval and a review decision; inspect rag_indexed/backend evidence. Reject records rejection and its decision. Neither sends payment, and reject does not explicitly revoke all earlier vectors.

# Challenge resolution and tricky follow-ups

## All four members

Problem -> code approach -> observable evidence -> remaining limit. Avoid unsupported claims of perfect resolution.

### Q51. How did you address cross-framework coordination?

Use named A2A skills and structured message/task results, with ADK Runner execution behind that boundary. Demonstrate a card, delegated request and actual executor. Remaining limits include custom protocol compatibility and fallback mode.

### Q52. How did you address inconsistent tool access?

Register shared tools and route discovery/invocation through the MCP client/dispatcher. Show metadata, arguments and result/error. Remaining limits include incomplete universal schema validation and direct node paths outside MCP.

### Q53. How did you address unsafe input and opaque execution?

Apply actual RAI scan calls on relevant boundaries and trace selected operations with Langfuse. Demonstrate a blocked override and a real nested trace. Neither regex controls nor tracing guarantees complete safety or privacy.

### Q54. How did you address premature indexing?

Insert a risk gate after reporting in the normal fresh-processing path. Auto-approved cases index; pending cases wait for explicit review. Verify status and indexing outcome, while qualifying direct-index and old-vector edge cases.

### Q55. What is the difference between no evidence and low-quality evidence?

No matching chunks means the corpus/filter may be empty, possibly because an invoice is pending review. Low quality means retrieved context or the answer failed evaluation. Refining wording cannot create evidence that was never indexed.

### Q56. What happens after RAG retries do not pass?

The current implementation can select its best-scoring attempt, label it quality-gate-not-met and run final output guardrails. It is not always an unconditional abstention. Check the returned warning and passed flag.

### Q57. Does max_retries=3 mean four total attempts here?

No. The implementation starts attempt at one and stops at attempt >= max_retries, so the default is up to three total attempts. It can stop earlier for a pass or no retrieved chunks.

### Q58. What if the UI works but AWS credentials are absent?

The current wrapper can use deterministic handlers instead of live Bedrock. Non-English text may be passed through with zero translation confidence. Verify actual mode; a responsive dashboard is not a live-model connectivity test.

### Q59. Can we say all ERP access always uses HTTP?

Not universally for this code. Normal requests use mock ERP HTTP, but the graph validator has a direct ERP-helper fallback. Explain the intended normal boundary and disclose the fallback instead of making an absolute claim.

### Q60. What should we do if the retained slide conflicts with source?

Explain the actual code and qualify the label. In particular, revalidation order and immutable-original claims in the preserved reference slide are too strong. Do not change your answer to an inaccurate statement merely to match a picture.

# Last-minute revision

### Four challenge resolutions

Framework coordination: A2A skills/tasks. Tool consistency: MCP registry/discovery. Safety and debugging: actual RAI scans plus exported traces. Premature indexing: normal-path risk gate and explicit reviewer decisions.

### Exact code facts

Normal graph: extractor -> translator -> parser -> validator -> business_validation -> reporter -> risk_gate -> conditional indexing. Revalidation endpoint: validator -> reporter -> compute_risk -> pending_review.

### Numbers to remember

Six services; seven registered cards; thirteen tools. Price tolerance 5%, quantity tolerance 0%. RAG default chunk size 900 characters, overlap 150, top_k 12, rerank up to 5. Triad >0.70; default up to three total attempts.

### Risk example

One ERROR contributes 0.60 and normally exceeds the 0.30 routing threshold. One WARNING contributes 0.30 and can auto-approve without a hard block. Weighted risk is not fraud probability.

### Do not claim

Universal official-protocol compliance, every node through MCP, all ERP access through HTTP, immutable full version history, all traces anonymous, or every successful task using a live model.

### Safe demo evidence

Directory/card -> actual task/executor -> tool/result -> safety finding -> trace -> reviewer/index outcome. Keep real credentials, private invoice details and destructive reset commands out of the rehearsal.

### Where to read

bootstrap.py; a2a/; adk_agents/; mcp_server/; tools/registry.py; graph/workflow.py; utils/guardrails.py; utils/risk_scoring.py; utils/observability.py; main.py; rag/pipeline.py; config/rules.yaml.

### Scope of this booklet

Based on the supplied source and specification comparison, not a fresh Nuvepro execution. The older separate implementation's test result does not apply. Historical authorship and measured performance improvements are not invented.